LOCKIN - A Minimalist Pomodoro Timer Application

HCI & Immersive Technologies - Interactive Prototype Code

COSC 402 - Current Trends and Topics in Computing

Lagman, Fervicmar D.

April 23, 2026

BSCS 4-5

Purpose and Objectives

LOCKIN is a browser-based Pomodoro productivity timer built with a modern React + TypeScript + Vite stack. The application implements the Pomodoro Technique, a time-management method that breaks work into focused intervals separated by short recovery periods — in a deliberately clean and distraction-free interface. The name "LOCKIN" reflects the application's core promise: helping users achieve and maintain a state of deep focus. The fundamental purpose of LOCKIN is to reduce cognitive overhead around time management during work or study sessions. Rather than tracking time mentally or switching between multiple tools, users are given a single-purpose interface that clearly communicates the current session mode, the time remaining, and the controls needed to manage it. The website was designed to satisfy these objectives:

- To present a clear countdown timer with three session types (Focus, Short Break, and Long Break) modeled after the classic 25-5-15 Pomodoro rhythm
- To eliminate all non-essential UI elements so the user's attention is anchored to the timer itself, reducing distraction
- To allow user to modify the duration of each session type through a modal-based settings panel, accommodating individual workflow preferences
- To provide a real-time visual feedback for all interactions, including effects on controls, live timer countdown, and modal overlay for settings

The website is also guided by design principles such as clarity over density to display exactly what the user needs at any given moment. Session mode tabs, a large timer display, session indicators, and playback controls are all that is visible on the main screen. Additionally, it has calm aesthetics, using a green color palette for its psychological associations with focus, calmness, and reduces stress. Lastly, it is accessible with large clickable buttons and icons, clean iconography, and labeled inputs to ensure that the interface is usable. The interface is deliberately generic; it makes no assumptions about what the user is working on, making it suitable across contexts.

System Design

The methodology of this study consists of technology stack, design, and application architecture of the prototype.

Technology Stack

The website is built on a modern JavaScript ecosystem using React, a component-based framework providing reactive state management and efficient DOM updates. Typescript is used to enforce type safety across props, state, and event handlers. To create the project, Vite is used which is a build tool for near-instant hot module replacement and optimized production bundles. Tailwind CSS is also employed in the implementation for rapid and consistent styling.

Design

The design of the website is created using Figma. The elements of the design are simple and minimalist to evoke calmness with its aesthetic. The font used is Britanica Font while the palette is based on color green, specifically, #1F4D2B.

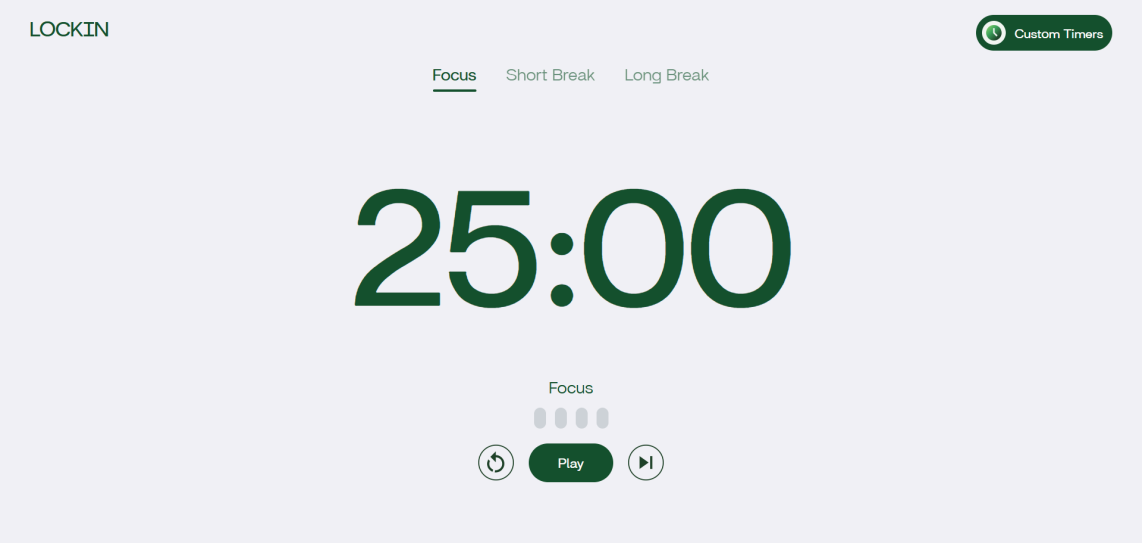

Figure 1. Landing Page - Timer

The landing page displays the timer and the necessary controls for the pomodoro timer. It presents a clean and distraction-free environment centered around a large and readable timer. Users can switch between Focus, Short Break, and Long Break modes using clearly labeled tabs. Primary controls are positioned below the timer for quick access, promoting efficient interaction. The minimalist layout, consistent color palette, and large typography reduce cognitive load and keep the user focused on the task.

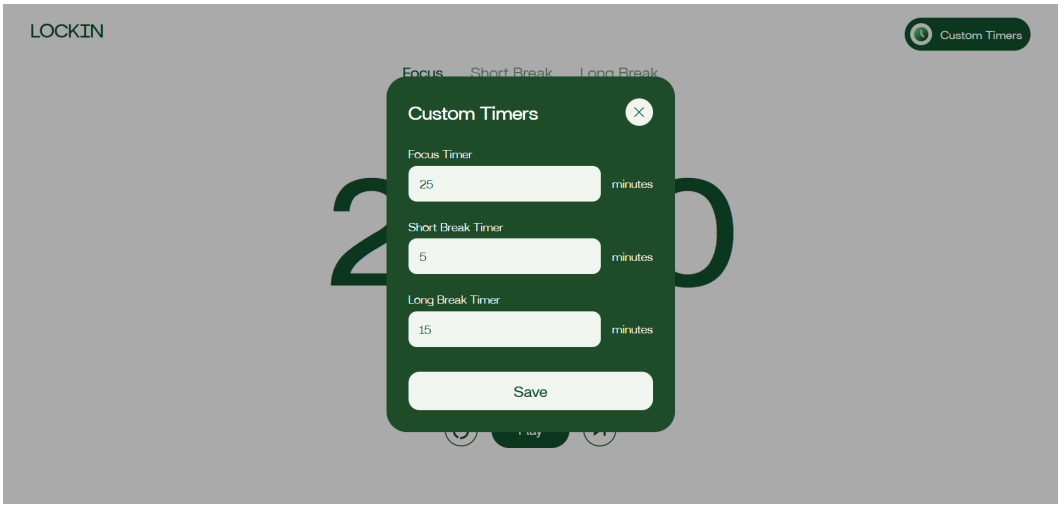

Figure 2. Custom Timers Modal

The Custom Timers modal allows users to personalize session durations without leaving the main interface. It uses a focused overlay to maintain context while enabling adjustments. Each input field is clearly labeled with units (“minutes”) to prevent confusion, and the layout ensures readability and ease of input. The prominent Save button emphasizes action completion, while the close button provides an easy exit, supporting user control and flexibility.

Application Architecture

LOCKIN is a Single Page Application (SPA) with a flat, component-based architecture. There is no routing layer as the application lives on a single view whose content is governed by React state. The component hierarchy is organized as follows:

- App.tsx (Root): Holds global state; passes data and callbacks down to child components as props.
- TimerDisplay: Receives the current time value and renders the clock
- ModeSelector: Buttons that send mode-change events to the root state.
- TimerControls: Play/Pause, Reset, and Skip buttons with corresponding icon assets. Triggers timer start/stop/reset via callbacks.
- SessionIndicator: Renders a row of dot indicators reflecting the current Pomodoro cycle position (e.g., 1 of 4 sessions completed).
- CustomTimerModal: Overlay component that conditionally renders based on modal state. Contains three labeled number inputs and a Save button that updates root-level duration state.

Implementation

Project Scaffold and Configuration

The project was initialized using the official Vite scaffolding tool with the React + TypeScript template. This produces a pre-configured environment with tsconfig.json, vite.config.ts, and the necessary ESLint baseline. Tailwind CSS was added as a PostCSS plugin.

Component Design Decisions

Implementation decisions for better organization and efficiency:

- Reusable Components: Buttons and inputs are factored into small, single-responsibility components so that changes to hover state or sizing propagate uniformly across all instances without duplication.
- Conditional Rendering: The modal is conditionally mounted so it does not occupy DOM space when inactive.
- TypeScript Prop Typing: All props are typed with TypeScript interfaces, preventing mismatched prop signatures across the component tree.
- Event Handling over Forms: Interactions are handled via onClick and onChange event handlers rather than native HTML form submission, avoiding unintended page reloads.

User Interaction

The main screen is composed of six visible interface regions, each serving a distinct function in the user's interaction flow:

UI Element	Function & Behavior
LOCKIN Logo (top-left)	Static brand identifier and not interactive.
Custom Timers Button (top-right)	Rounded pill button with a clock icon. On click, it renders CustomTimerModal overlay.
Mode Tabs (center-top)	Three text tabs: Focus, Short Break, Long Break. The active tab is underlined with a green indicator. Clicking a tab switches mode and resets the timer.
Timer Display (center)	Large MM:SS countdown, updating every second when running.

Session Label & Dots	Text label showing current mode name below the timer. Four dot indicators show progress through the Pomodoro cycle.
Playback Controls	Three controls: Reset (circular arrow icon), Play/Pause (pill button, primary CTA), Skip (fast-forward icon). Reset and Skip are circle-outlined icon buttons.

The interaction design is guided by six core UX principles. Minimalism ensures that every element serves a purpose, avoiding distractions like unnecessary visuals. Visual hierarchy emphasizes the timer as the primary focus, with controls and tabs placed as secondary elements. Consistency is maintained through a unified green palette, Britanica typeface, and rounded components. Feedback is provided through responsive interactions, real-time updates, and modal overlays that preserve context. Affordance is achieved using familiar icons and clearly labeled inputs, making functions intuitive. Lastly, accessibility is supported through readable contrast, visible labels, and appropriately sized interactive elements.

Testing and Results

The interaction design is guided by six core UX principles. Minimalism ensures that every element serves a purpose, avoiding distractions like unnecessary visuals. Visual hierarchy emphasizes the timer as the primary focus, with controls and tabs placed as secondary elements. Consistency is maintained through a unified green palette, Britanica typeface, and rounded components. Feedback is provided through responsive interactions, real-time updates, and modal overlays that preserve context. Affordance is achieved using familiar icons and clearly labeled inputs, making functions intuitive. Lastly, accessibility is supported through readable contrast, visible labels, and appropriately sized interactive elements.

Challenges and Future Improvements

Challenges

Several challenges were encountered during development and addressed accordingly. For font loading, incorrect relative paths in @font-face caused fonts to fail in production, which was resolved by using absolute paths. setInterval drift was mitigated by anchoring timer updates to Date.now() to maintain accuracy. In terms of modal and timer interaction, the timer was allowed to continue running while the settings modal is open to preserve user flow.

Future Improvements

The current implementation serves as a complete MVP, with several high-value improvements identified for future iterations. Adding localStorage persistence would allow custom settings and session progress to survive page reloads. Integrating the Web Notifications API would notify users when sessions end, even in background tabs. Implementing automatic Pomodoro cycles would streamline transitions between focus and break periods. Enhancing audio customization would let users choose or upload notification sounds. A statistics dashboard could track sessions, focus time, and streaks to boost motivation. Finally, converting the app into a Progressive Web App (PWA) would enable offline use and a more native-like experience on desktop and mobile.

Conclusion

LOCKIN demonstrates that a disciplined constraint, "show only what the user needs", produces a more effective tool than feature-maximalist alternatives. By combining React's component model with TypeScript's type safety, Vite's build performance, and Tailwind's utility styling, the application achieves its core purpose with a lean, maintainable codebase. The green aesthetic and typographic choices reinforce the product's identity as a calm, focused companion for deep work. Future iterations will extend functionality without compromising the minimalist foundation that defines the application's value.